Mini Lisp

User Guide, Language Reference, and Implementation Notes

Python + pyparsing / SKED FACTORIAL-GATE

Technical documentation based on the supplied source implementation

13 October 2026 | Version 0.1

Abstract

This manual documents a deliberately small Lisp interpreter written in Python with a `pyparsing` front end. It covers setup, executing source programs, the exact implemented language subset, API usage, diagnostics, validation, and the implementation design. The implementation notes explicitly distinguish existing behavior from proposed future improvements. This is *not* a Scheme or Common Lisp conformance claim.

Contents

1 Purpose and Scope **2**
- 1.1 Project goal . . . 2
- 1.2 What this version does . . . 2
- 1.3 What this version is not . . . 2
- 1.4 Repository structure . . . 2

2 Installation and First Execution **4**
- 2.1 Requirements . . . 4
- 2.2 Creating the environment . . . 4
- 2.3 Run the factorial example . . . 4
- 2.4 Run automated tests . . . 4
- 2.5 Run a modified program . . . 5

3 Writing Programs in Mini Lisp **6**
- 3.1 The factorial program . . . 6
- 3.2 Syntax at a glance . . . 6
- 3.3 Integer literals . . . 6
- 3.4 Symbols . . . 6
- 3.5 Definitions . . . 6
- 3.6 Anonymous functions and calls . . . 7
- 3.7 Conditional expressions . . . 7
- 3.8 Primitive functions . . . 7
- 3.9 Several top-level expressions . . . 7
- 3.10 Lexical closures . . . 8
- 3.11 Examples of unsupported syntax . . . 8

4 Execution Model and Language Semantics **9**
- 4.1 Expression categories . . . 9
- 4.2 Evaluation order . . . 9
- 4.3 Function recursion . . . 9
- 4.4 Factorial trace . . . 10
- 4.5 Simplified grammar . . . 10

5 Command-Line and Python Interfaces **11**
- 5.1 Command-line interface . . . 11
- 5.2 Parser API . . . 11
- 5.3 Runtime API . . . 11
- 5.4 Environment API . . . 12

6 Errors and Troubleshooting **13**
6.1 Error classes and reporting . . . 13
6.2 Common user mistakes . . . 13
6.2.1 Attempting to use addition . . . 13
6.2.2 Calling a function before defining it . . . 13
6.2.3 Passing a negative factorial input . . . 13
6.2.4 Assuming zero is false . . . 14
6.3 Diagnostic limitations . . . 14

7 Validation and SKED Acceptance **15**
7.1 FACTORIAL-GATE contract . . . 15
7.2 Test commands and reproducibility . . . 15
7.3 Acceptance versus future gates . . . 15

8 Implementation Nuances and Engineering Rationale **17**
8.1 Why pyparsing is used . . . 17
8.2 Parser construction . . . 17
8.3 Token precedence and atom boundaries . . . 17
8.4 AST representation is deliberately lightweight . . . 18
8.5 The Symbol design and a subtle type property . . . 18
8.6 Environment lookup and shadowing . . . 18
8.7 Closure value representation . . . 18
8.8 How self-recursion succeeds . . . 18
8.9 Special forms versus normal functions . . . 18
8.10 Subtle distinction between equality and truth . . . 19
8.11 Built-in function invocation and exceptions . . . 19
8.12 Variadic primitive details . . . 19
8.13 Interpreter recursion and stack consumption . . . 19
8.14 Memory management and reference cycles . . . 19
8.15 Error boundaries and security considerations . . . 19
8.16 Why `run` returns only the final value . . . 20
8.17 No dedicated source-location diagnostics . . . 20
8.18 Host-language coupling . . . 20

9 Evolution Roadmap **21**
9.1 Minimal next iteration . . . 21
9.2 Intermediate language features . . . 21
9.3 Longer-term architecture . . . 21
9.4 Suggested SKED artifacts . . . 21

A Complete Source Example **23**

B Quick Reference **24**

C Document Basis **25**

Chapter 1

Purpose and Scope

1.1 Project goal

Mini Lisp is a teaching and engineering prototype that demonstrates the execution path from S-expression source text to a result. Its immediate milestone is the SKED **FACTORIAL-GATE**: parse a Lisp file, define a recursive factorial function, execute it, and print the answer `120` for input `5`.

1.2 What this version does

The existing code implements integer literals, symbols, parenthesized expressions, semicolon-to-end-of-line comments, top-level sequences, `define`, `lambda`, `if`, function application, lexical closures, and three primitives `=`, `-`, and `*`. It reads programs from UTF-8 files and returns the value of the last top-level expression.

1.3 What this version is not

It does not implement a REPL, ordinary Lisp linked lists, quoted forms, string literals, general-purpose numeric types, macros, `let`, `begin`, Boolean literals, user-facing input/output functions, bytecode, tail-call optimization, or a separate garbage collector. The Python interpreter supplies the memory manager. Neither untrusted-code isolation nor resource quotas are provided.

1.4 Repository structure

```
mini_lisp_sked/
  README.md
  requirements.txt
  parser.py
  runtime.py
  main.py
  examples/
    factorial.lisp
  tests/
    test_factorial.py
```

File	Responsibility
`parser.py`	pyparsing grammar, `Symbol` node type, and `parse(source)`.

File	Responsibility
`runtime.py`	lexical environments, closure values, special forms, primitive operations, evaluation and program execution.
`main.py`	command-line file loading and user-visible errors.
`examples/factorial.lisp`	canonical factorial program.
`tests/test_factorial.py`	unit tests used by the current milestone.
`requirements.txt`	pyparsing version constraint `>=3.2,<4`.

Chapter 2

Installation and First Execution

2.1 Requirements

Use Python 3 with `venv` and a version of `pyparsing` satisfying the project's dependency declaration. Python 3.10 or later is a reasonable development target, but the repository does not currently enforce an exact Python minimum. The examples below use a standard POSIX shell as found on Debian 13.

2.2 Creating the environment

Run these commands from the project root.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

The venv isolates project dependencies from the system interpreter. To check the selected interpreter and installed library:

```
python --version
python -m pip show pyparsing
```

2.3 Run the factorial example

```
python main.py examples/factorial.lisp
```

Expected standard output:

```
120
```

A successful run exits with status code zero. Parsing, evaluation, recursion-limit, and file-reading failures are reported to standard error with a nonzero exit status.

2.4 Run automated tests

```
python -m unittest discover -s tests -v
```

The repository contains six `unittest` methods; one includes five separate factorial input cases. A passing result validates the selected cases, not language completeness.

2.5 Run a modified program

Copy `examples/factorial.lisp` to another file, then alter the last expression:

```
cp examples/factorial.lisp examples/factorial10.lisp
```

Change `(factorial 5)` to `(factorial 10)` and run:

```
python main.py examples/factorial10.lisp
```

Expected output: `3628800`. The interpreter does not accept a Lisp expression directly as a command-line argument; its sole positional argument is a filename.

Chapter 3

Writing Programs in Mini Lisp

3.1 The factorial program

```
; A comment occupies the rest of this line.
(define factorial
  (lambda (n)
    (if (= n 0)
        1
        (* n (factorial (- n 1))))))

(factorial 5)
```

The first top-level expression binds `factorial` to a closure; the second calls that closure. The script prints only the result of the last expression.

3.2 Syntax at a glance

Programs use prefix notation and parentheses to denote expression structure. For example `(- 7 2)` computes five and `(* 2 3 4)` computes 24. Whitespace separates atoms; newlines have no additional statement-ending meaning. A semicolon starts a comment that continues to the end of its line.

3.3 Integer literals

Supported numeric literals are signed decimal integers, for example `0`, `17`, `-3`, and `+12`. Floating-point and hexadecimal tokens are not interpreted as numbers. They may be lexed as symbols and fail at evaluation unless they have bindings.

3.4 Symbols

A symbol names a variable, primitive function, or special form. Examples include `n`, `factorial`, `*`, and `=`. Symbols are case-sensitive. A symbol cannot contain whitespace, `(`, `)`, or `;`. There are no string literals, even though symbols are internally represented by a subclass of Python `str`.

3.5 Definitions

```
(define answer 42)
answer
```

`define` requires exactly an identifier and an expression; this version does not support the Scheme shorthand `(define (f x) ...)`. A definition updates the current environment and evaluates to its assigned value.

3.6 Anonymous functions and calls

```
(define square (lambda (x) (* x x)))
(square 6)
```

A `lambda` has one parameter list and exactly one body expression. Parameter lists may be empty; duplicate parameter names are rejected. A call evaluates its operator, then its arguments from left to right, then invokes the resulting callable. The number of arguments to a user-defined function must match the number of parameters exactly.

3.7 Conditional expressions

```
(if (= 2 2) 10 20)
```

`if` requires exactly three subexpressions: condition, then branch, and else branch. Only the chosen branch is evaluated. The implementation selects the else branch *only* when the condition is Python `False`; all other values, including the integer `0`, count as true. This truth rule matches the idea that zero is not false, but no source-level Boolean literals are currently supplied.

3.8 Primitive functions

Form	Meaning	Example / result
`=`	Exactly two integer arguments; returns a Python Boolean internally.	`(= 3 3)` → `True`
`-`	One or more integer arguments; unary negation or left operand minus the sum of remaining operands.	`(- 10 2 3)` → `5`
`*`	Zero or more integer arguments; their product; empty product is one.	`(* 2 3 4)` → `24`

Both `-` and `*` enforce integer arguments through `expect_integer`; `=` does likewise. Python `bool` values are explicitly excluded from integer operands even though `bool` subclasses `int` in Python. `(+ 2 3)` is *not* implemented.

3.9 Several top-level expressions

```
(define a 8)
(define b 3)
(- a b)
```

This file returns `5`. Top-level forms execute in source order. An empty file is accepted, and `run` returns Python `None`; `main.py` therefore prints `None`.

3.10 Lexical closures

```
(define make-subtractor
  (lambda (x)
    (lambda (y) (- x y))))

((make-subtractor 7) 2)
```

The result is `5`. The inner closure retains access to the environment where it was created, so `x` continues to refer to seven after the outer call has returned.

3.11 Examples of unsupported syntax

```
'hello                  ; quote shorthand is not supported
(quote hello)           ; quote special form is not supported
(+ 1 2)                 ; '+' primitive is absent
(lambda (x) x x)        ; more than one body expression
(define (f x) x)        ; function shorthand is absent
```

The first example may be parsed as a symbol and fail on lookup; others may parse successfully but fail during evaluation. Syntax acceptance does not imply semantic support.

Chapter 4

Execution Model and Language Semantics

4.1 Expression categories

The parser yields Python `int` values, `Symbol` values, and nested Python `list` values. The evaluator maps integers to themselves, resolves symbols through an environment, recognizes the three special forms, and otherwise treats a nonempty list as a function call. Empty list expressions are rejected during evaluation.

4.2 Evaluation order

1. A literal integer evaluates to itself.
2. A symbol resolves to its nearest visible binding.
3. `define`, `lambda`, and `if` receive dedicated evaluation rules.
4. For an ordinary call, the operator expression is evaluated first.
5. Each argument is evaluated in left-to-right source order.
6. User-defined function calls create a local environment extending the closure's captured environment.
7. The function body executes in that local environment and its value becomes the call result.

4.3 Function recursion

The factorial closure captures the global environment object when `lambda` is evaluated. The following `define` then installs the new closure in that *same mutable environment*. Recursive calls find `factorial` there. No special recursive-function syntax or fixed-point combinator is required.

4.4 Factorial trace

$$\begin{aligned} f(5) &= 5 \cdot f(4) \\ &= 5 \cdot 4 \cdot f(3) \\ &= 5 \cdot 4 \cdot 3 \cdot f(2) \\ &= 5 \cdot 4 \cdot 3 \cdot 2 \cdot f(1) \\ &= 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot f(0) \\ &= 120. \end{aligned}$$

The base case `(= n 0)` is evaluated before either branch; at zero, `if` returns the integer one without recursing further.

4.5 Simplified grammar

The following is an explanatory EBNF-like sketch, not an exact replacement for pyparsing's token-matching and comment-handling behavior.

```
program    := expression*
expression := integer | symbol | '(' expression* ')'
integer    := ['+' | '-'] decimal_digit+
symbol     := one_or_more_nonwhitespace_non_delimiter_chars
comment    := ';' all_chars_until_newline
```

Syntactic correctness at this level is deliberately broader than meaningful executable forms. The evaluator enforces shapes for special forms and nonempty calls.

Chapter 5

Command-Line and Python Interfaces

5.1 Command-line interface

```
python main.py PATH_TO_SOURCE.lisp
```

`main.py` uses `argparse` and `pathlib.Path`, reads the input as UTF-8, parses all content, executes the resulting expressions, and prints the final result via Python `print`. There is no interactive mode and no other application-specific flags.

5.2 Parser API

```
from parser import parse, Symbol
expressions = parse('(define n 5) (* n 2)')
# [[Symbol('define'), Symbol('n'), 5],
#  [Symbol('*'), Symbol('n'), 2]]
```

`parse(source: str)` processes the entire input, returns a list of top-level expressions, and raises a pyparsing parsing exception if the input is malformed. Text after a valid program is not silently accepted.

5.3 Runtime API

```
from parser import parse
from runtime import run, standard_environment

program = parse('(define n 5) (* n 2)')
result = run(program)
assert result == 10

env = standard_environment()
assert run(parse('(define x 7)'), env) == 7
assert run(parse('(* x 6)'), env) == 42
```

`run(expressions, env=None)` constructs a fresh standard environment when `env` is omitted. Supplying a shared environment lets consecutive calls retain bindings. `evaluate(expr, env)` executes a single expression and is also public at the Python module level, although no stable API guarantee is declared.

5.4 Environment API

```
from runtime import Environment
root = Environment({'x': 10})
child = Environment({'y': 3}, parent=root)
assert child.get('x') == 10
assert child.get('y') == 3
child.define('x', 8)  # shadows root binding
```

An unresolved name raises `LispError`. Keys typically are `Symbol` instances, which compare and hash like regular strings since `Symbol` derives from `str`.

Chapter 6

Errors and Troubleshooting

6.1 Error classes and reporting

Situation	Expected behavior
Unmatched parenthesis	pyparsing parsing exception caught by CLI.
Unknown name	`LispError: Undefined symbol: ...`.
Malformed special form	`LispError` with a form-specific syntax hint.
Wrong user-closure arity	`LispError` with expected and actual argument counts.
Wrong arithmetic input type	`LispError: Expected integer, got ...`.
Wrong `=` arity	`LispError` requiring exactly two arguments.
Empty `(-)`	`LispError` requiring at least one argument.
Missing file or invalid UTF-8	`OSError` is caught for filesystem errors; Unicode decoding errors are *not* caught by the current CLI handler.
Deep or nonterminating recursion	Python `RecursionError` caught by the CLI.

6.2 Common user mistakes

6.2.1 Attempting to use addition

```
(+ 1 2)
```

This results in an undefined-symbol error because `+` has no standard-environment binding.

6.2.2 Calling a function before defining it

```
(factorial 5)
(define factorial (lambda (n) ...))
```

The first form fails: top-level evaluation is sequential and the function has not been bound.

6.2.3 Passing a negative factorial input

```
(factorial -1)
```

The example definition handles zero but never reaches it when starting from a negative number and subtracting one repeatedly. The resulting recursion error is *not* a proper domain validation

response. The factorial function's domain should be restricted by a future implementation change, or the user should supply nonnegative integers only.

6.2.4 Assuming zero is false

```
(if 0 11 22)
```

The result is `11`: only the Python value `False` triggers the else branch.

6.3 Diagnostic limitations

There is no persistent source-location annotation in AST nodes, stack trace of Lisp calls, explicit evaluation-step budget, or purpose-built error reporting for all Python primitive exceptions. The CLI catches the listed exceptions but does not guarantee that every possible runtime failure becomes a clean Lisp diagnostic.

Chapter 7

Validation and SKED Acceptance

7.1 FACTORIAL-GATE contract

The baseline acceptance contract is successful file execution for nonnegative integer factorial inputs, correct basic arithmetic and scoping, predictable rejection of malformed input, and no evaluation of the unselected `if` branch. No negative-input domain protection is claimed.

Case	Input	Expected output / property
Base case	`(factorial 0)`	`1`
Small recursion	`(factorial 1)`	`1`
Recursion	`(factorial 3)`	`6`
Canonical example	`(factorial 5)`	`120`
Larger example	`(factorial 10)`	`3628800`
Lexical scoping	`((make 7) 2)` after defining `make`	`5`
Conditional laziness	`(if (= 0 0) 1 unknown)`	`1`
Undefined symbol	`(factorial x)`	`LispError`
Malformed expression	`(factorial 5`	Parse error

7.2 Test commands and reproducibility

```
python -m unittest discover -s tests -v
python main.py examples/factorial.lisp
```

For reproducibility, record Python version, `pyparsing` version, operating system, test results and the source revision. The dependency currently permits a range of pyparsing versions; a fully locked requirements file is not yet included.

7.3 Acceptance versus future gates

- **FACTORIAL-GATE** tests the implemented minimal interpreter.
- **DOMAIN-GATE** (proposed) rejects negative inputs and invalid argument types with explicit domain errors.
- **ROBUSTNESS-GATE** (proposed) specifies limits, reliable diagnostics, parser edge cases, and exception containment.

- **LANGUAGE-EXTENSION-GATE** (proposed) introduces more forms and primitives without changing existing behavior unexpectedly.

Chapter 8

Implementation Nuances and Engineering Rationale

This chapter describes implementation details taken directly from `parser.py`, `runtime.py`, and `main.py`. Where an improvement is discussed, it is marked as *proposed*, not as an implemented feature.

8.1 Why pyparsing is used

The front end delegates recursive syntax recognition and token grouping to `pyparsing`, avoiding a hand-written lexer and recursive-descent parser. The parser declares a `Forward` expression placeholder, makes a parenthesized group refer back to that expression, and sets the placeholder to the alternation of lists and atomic values. This supports arbitrary syntactic nesting, subject to Python and parser-library resource limits.

8.2 Parser construction

The essential implementation is:

```
expr = Forward()
integer = Regex(r"[+-]?\d+(?![^\s()])").set_parse_action(
    lambda t: int(t[0]))
symbol = Regex(r"[^\s();]+(?![^\s()])").set_parse_action(
    lambda t: Symbol(t[0]))
atom = integer | symbol
sexpr = Group(Suppress("(") + ZeroOrMore(expr) + Suppress(")"))
expr <<= (sexpr | atom)
program = ZeroOrMore(expr)
program.ignore(Regex(r";[^\n]*"))
```

The implementation does not include a separate token stream object or tokenizer class. `Suppress` discards parentheses, `Group` retains nested expression structure, and `parse_string(..., parse_all=True).as_` converts the result to nested Python lists.

8.3 Token precedence and atom boundaries

The integer pattern is tried before the symbol pattern. Its final negative lookahead is intended to avoid accepting the number `123` as the prefix of `123abc`. The symbol pattern prohibits whitespace, parentheses, and semicolons. It is a pragmatic small-language grammar rather than

a formal Scheme tokenization standard. Edge cases and any changes to the regular expressions should be protected by regression tests.

8.4 AST representation is deliberately lightweight

The syntax tree is a nested Python list rather than a hierarchy of AST classes. An integer atom is `int`; an identifier is `Symbol`, a `str` subclass; and a parenthesized expression is a `list`. This minimizes code, but gives up source spans, explicit node kinds, easy structural validation, and immutable syntax nodes. A future location-aware AST would require a conversion layer or revised parser actions.

8.5 The Symbol design and a subtle type property

Since `Symbol` inherits from `str`, `Symbol('x') == 'x'` is true. This allows the current runtime to key dictionaries and compare special-form names conveniently. The evaluator tests symbols using `isinstance(expr, Symbol)`, so arbitrary Python strings passed directly to `evaluate` are not valid Lisp identifiers. Avoid bypassing the parser unless constructing valid `Symbol` nodes explicitly.

8.6 Environment lookup and shadowing

`Environment` holds a mutable `bindings` dictionary and an optional `parent` link. Lookup searches the local dictionary first, then follows parents recursively; a local `define` writes only to the current dictionary. This is lexical scope, not dynamic scope, because closures retain their definition-site environment, not the caller's environment.

8.7 Closure value representation

```
@dataclass
class Closure:
    params: list
    body: list
    env: Environment
```

A closure stores formal parameter names, a single body AST, and the environment captured at lambda creation. Calling it creates a child environment containing formal-to-actual argument mappings. The Python evaluator recursively evaluates the stored body.

8.8 How self-recursion succeeds

`(define factorial (lambda ...))` evaluates `lambda` first, capturing the current global `Environment` object. It then installs `factorial` into that same object. Because the closure holds a reference, not a frozen copy, its body can look up its own name. Rebinding the name later changes what recursive lookup finds. This is a consequence of mutable lexical environments and should be documented when expanding the language.

8.9 Special forms versus normal functions

`define`, `lambda`, and `if` are selected by examining the unevaluated first element of a nonempty list. They cannot be implemented simply as strict ordinary functions here, because definitions

need an unevaluated name, lambdas need an unevaluated body, and `if` must avoid evaluating both branches. Special-form dispatch has priority over environment lookup, so user bindings cannot override these syntactic names when they appear directly as list heads.

8.10 Subtle distinction between equality and truth

The equality primitive returns Python `True` or `False`. The evaluator accepts internal Booleans as literal values if called programmatically. But the parser has no Boolean literals, and the `if` check uses `condition is not False`. In particular, numeric zero is true; Python `None` would also be regarded as true if introduced through custom host bindings. A formal Boolean type would make the semantics less dependent on Python implementation details.

8.11 Built-in function invocation and exceptions

An ordinary call evaluates its operator to a value. If the value is a `Closure`, the interpreter checks arity and evaluates the body in a child scope. If it is any Python-callable object, it calls it directly. The host-call branch catches `TypeError` and wraps it as `LispError`; other host exceptions may escape. This is useful for prototyping, but it is not a restricted native-function interface. *Proposed improvement*: maintain an explicit whitelist or primitive descriptor registry with arity, argument type contracts, and normalized exceptions.

8.12 Variadic primitive details

`multiply(*args)` multiplies all integer operands via `functools.reduce` with identity `1`, making `(*)` return one. `subtract(*args)` negates its single argument or subtracts the sum of all trailing arguments from the first. `equal(*args)` checks exactly two arguments and compares integers. These are subset-specific semantics and should not be extrapolated to full Scheme numeric behavior.

8.13 Interpreter recursion and stack consumption

Evaluation relies on recursive Python function calls, including nested calls to `evaluate` during user-function invocation. Consequently, factorial of a sufficiently large input may fail with `RecursionError`, even though Python supports arbitrarily large integers. There is no trampoline, continuation machine, tail-call elimination, or execution-step limit. *Proposed improvement*: use an explicit evaluation loop or a small-step abstract machine for better control over recursion and quotas.

8.14 Memory management and reference cycles

No project-specific allocator or garbage collector is defined. Objects are created and managed by Python. Closures can capture environments that reference those closures through bindings, making cyclic object graphs possible. Python's cyclic garbage collector normally handles unreachable cycles, but the interpreter does not provide explicit memory ceilings, deterministic lifetime, or graph inspections.

8.15 Error boundaries and security considerations

The CLI catches `OSError`, `ParseBaseException`, `LispError`, and `RecursionError`. The parser rejects malformed trailing text through `parse_all=True`. This does not make the interpreter a

sandbox. An untrusted program can consume CPU and memory, and a host application can inject arbitrary Python callables into the environment. File access is provided by the command-line host, not by built-in Lisp functions. Do not execute untrusted inputs without external process isolation and resource limits.

8.16 Why `run` returns only the final value

`run` loops over each parsed top-level expression, replacing its `result` variable, and then returns the last result. This naturally supports `define` followed by an invocation and keeps the CLI trivial. It also means intermediate results are not displayed. A REPL would need its own read/print policy and possibly a persistent environment.

8.17 No dedicated source-location diagnostics

Because the parsed AST consists only of Python values and lists, source line and column positions are discarded before evaluation. Parse errors can reference source positions through pyparsing, but an undefined symbol deep in a function has no retained AST source location. *Proposed improvement*: preserve spans in the parser output and include a Lisp call stack in runtime exceptions.

8.18 Host-language coupling

The current implementation deliberately uses Python integers, Booleans, lists, dicts, dataclasses, exceptions, callables, and garbage collection. This reduces code size and is excellent for initial SKED validation. It also means behavior such as recursion depth, error wording, Boolean printing (`True`/`False`), and integer representation remains coupled to Python rather than defined by an independent Lisp runtime.

Chapter 9

Evolution Roadmap

9.1 Minimal next iteration

Before adding broad language features, introduce negative-factorial domain protection, explicit integer and callable argument validation at public boundaries, source position reporting, and regression tests for empty lists and malformed atom boundaries. Distinguish language-defined errors from host exceptions. Keep `FACTORIAL-GATE` passing.

9.2 Intermediate language features

Possible extension order: addition and comparison primitives; Boolean constants; local binding via `let`; a multi-expression `begin`; first-class pairs and lists; quotation; basic predicates and formatted output. Each extension needs documented syntax, evaluation semantics, and independent acceptance tests.

9.3 Longer-term architecture

If the aim becomes a general interpreter rather than a compact experiment, consider a stable typed AST, token spans, environment abstraction, primitive registration layer, trampoline/explicit machine, deterministic test fixtures, memory and CPU resource ceilings, and integration into a continuous integration workflow.

9.4 Suggested SKED artifacts

Artifact	Content
Language subset specification	Grammar, value types, special forms and primitive contracts.
Architecture decision record	Justification for pyparsing, Python AST lists and recursive evaluator.
Acceptance test specification	Gate IDs, positive and negative fixtures, expected diagnostics.
Implementation inventory	Source files and function responsibilities, dependencies and version policy.
Risk register	Nontermination, Python recursion limits, host callable leakage, resource exhaustion.

Artifact	Content
Change log	Compatibility-sensitive syntax and semantic changes between versions.

Appendix A

Complete Source Example

```
; examples/factorial.lisp
(define factorial
  (lambda (n)
    (if (= n 0)
        1
        (* n (factorial (- n 1))))))

(factorial 5)
```

Appendix B

Quick Reference

Form	Behavior
`123`, `-7`	Integer values.
`name`	Look up a symbol.
`(define name expr)`	Bind in current environment.
`(lambda (x y) expr)`	Construct lexical closure.
`(if test yes no)`	Evaluate condition and one branch.
`(f arg1 arg2)`	Evaluate operator and arguments, then call.
`(= a b)`	Integer equality.
`(- x ...)`	Integer subtraction or negation.
`(* x ...)`	Integer multiplication.
`; comment`	Ignore through end of line.

Appendix C

Document Basis

This manual was prepared against the actual files distributed in the `mini_lisp_sked` project archive on 13 October 2026. Operational descriptions refer to the observed code structure in `parser.py`, `runtime.py`, `main.py`, `tests/test_factorial.py`, and the example source. Proposed features are not implied to exist in the supplied repository.